

ABAP Exceptions

Adrian Streitz, Johannes Rank, Borys Levkovskyi



Prerequisites

- Experience with ABAP Workbench and Eclipse
- Basic ABAP programming skills
- Basic knowledge in ABAP object-oriented programming

Agenda

- I. Introduction**
- II. Exception classes**
- III. Exception handling**
- IV. Exception raising**
- V. Global exception classes**
- VI. Propagation of exceptions**
- VII. Detailed exception handling**
- VIII. Check your knowledge**

I. Introduction

Motivation

Runtime error – ABAP program / SAP Gui

Runtime Error - Description of Exception

Long Text | Debugger

Runtime Errors	COMPUTE_INT_ZERODIVIDE
Except.	CX_SY_ZERODIVIDE
Date and Time	04.12.2008 11:56:32

Short text
Division by 0 (type I)

What happened?
Error in the ABAP Application Program

The current ABAP program "SAPLZ_00_FM_CALCULATION" had to be terminated because it has come across a statement that unfortunately cannot be executed.

Error analysis
An exception occurred that is explained in detail below.
The exception, which is assigned to class 'CX_SY_ZERODIVIDE', was not caught and therefore caused a runtime error.
The reason for the exception is:
In the current program "SAPLZ_00_FM_CALCULATION", an arithmetic operation ('DIVIDE', '/', 'DIV', or 'MOD') attempted to use operands of type I to divide by 0.

Missing Handling of System Exception

Program	ZZ_00_CALCULATION
---------	-------------------

Trigger Location of Exception

Program	SAPLZ_00_FM_CALCULATION
Include	LZ_00_FM_CALCULATIONU01
Row	23
Module type	(FUNCTION)
Module Name	Z_00_FM_CALCULATION

Runtime error – BSP

ZZA_WD_1ST_APP [Web Dynpro for ABAP] - Windows Internet Explorer

http://06.informatik.tu-muenchen.de:8000/sap/bc/webdynpro/sap/zza_wd_1st_app?sap-language=EN

Google

Los geht's! | Lesezeichen | 0 blockiert

Einstellungen | lenovo

ZZA_WD_1ST_APP [Web Dynpro for ABAP]

Seite | Extras

Error when processing your request

What has happened?

The URL http://see06.informatik.tu-muenchen.de:8000/sap/bc/webdynpro/sap/zza_wd_1st_app/ was not called due to an error.

Note

- The following error text was processed in the system S00 : **User session (HTTP/SMTP/...) closed after timeout**
- The error occurred on the application server see06_S00_00 and in the work process 0 .
- The termination type was: ERROR_MESSAGE_STATE
- The ABAP call stack was:
Module: %_HTTP_START of program SAPMHTTP

What can I do?

- If the termination type was RABAX_STATE, then you can find more information on the cause of the termination in the system S00 in transaction ST22.
- If the termination type was ABORT_MESSAGE_STATE, then you can find more information on the cause of the termination on the application server see06_S00_00 in transaction SM21.
- If the termination type was ERROR_MESSAGE_STATE, then you can search for more information in the trace file for the work process 0 in transaction ST11 on the application server see06_S00_00 . In some situations, you may also need to analyze the trace files of other work processes.
- If you do not yet have a user ID, contact your system administrator.

Error code: ICF-IE-http -c: 001 -u: LÜBECK -l: E -s: S00 -t: see06_S00_00 -w: 0 -d: 20081204 -t: 123951 -v: ERROR_MESSAGE_STATE -e: User session (HTTP/SMTP/...) closed after timeout

HTTP 500 - Internal Server Error
Your SAP Internet Communication Framework Team

What is an exception?

An **exception** is a situation that arises while an ABAP program is being executed, in which the continuation of the program is not possible, e.g. division by zero. Without exception handling, the complete stack is rolled back and the error is presented to the user.

With **exception handling** you can

- present the error to the user in a more user-friendly way
- react on the error
- cleanup (free any used resources before terminating)
- continue executing the program ignoring the error.

What is an exception?

Class-based exception concept

- SAP Web AS 6.10 introduced a new ABAP Objects exception concept that exists parallel to the existing concept based on sy-subrc.
- In this context, an exception is represented by an exception object (= instance of an exception class). The attributes of the exception(-object) contain details about the error which has occurred (e.g. an exception text).

Characteristics of ABAP exceptions

- Exceptions can be raised either implicitly in the ABAP runtime environment or explicitly in the ABAP program.
- Two main types of exceptions exist: those that can be **handled** and those that **cannot be handled**.

II. Exception classes

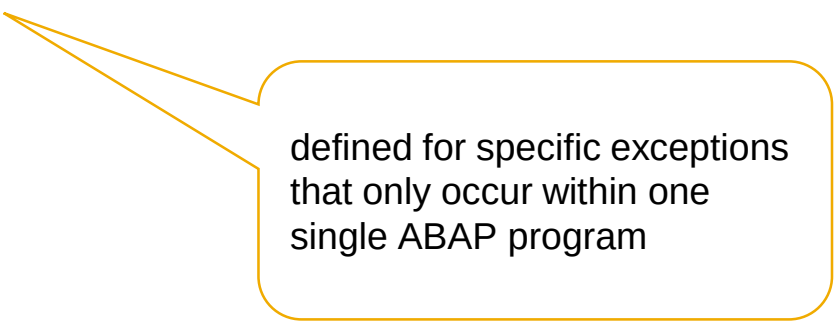
3 types of exception classes

- Predefined system exception classes '**CX_...**'
- Globally defined exception classes '**<namespace>CX_...**'.

Global exception classes are defined within the Class Builder in transaction SE24.

The assignment of the prefix '**<namespace>CX_...**' to a global exception class will have the effect, that the *class-type "Exception Class"* is already selected.

- Locally defined exception classes '**LCX_...**'

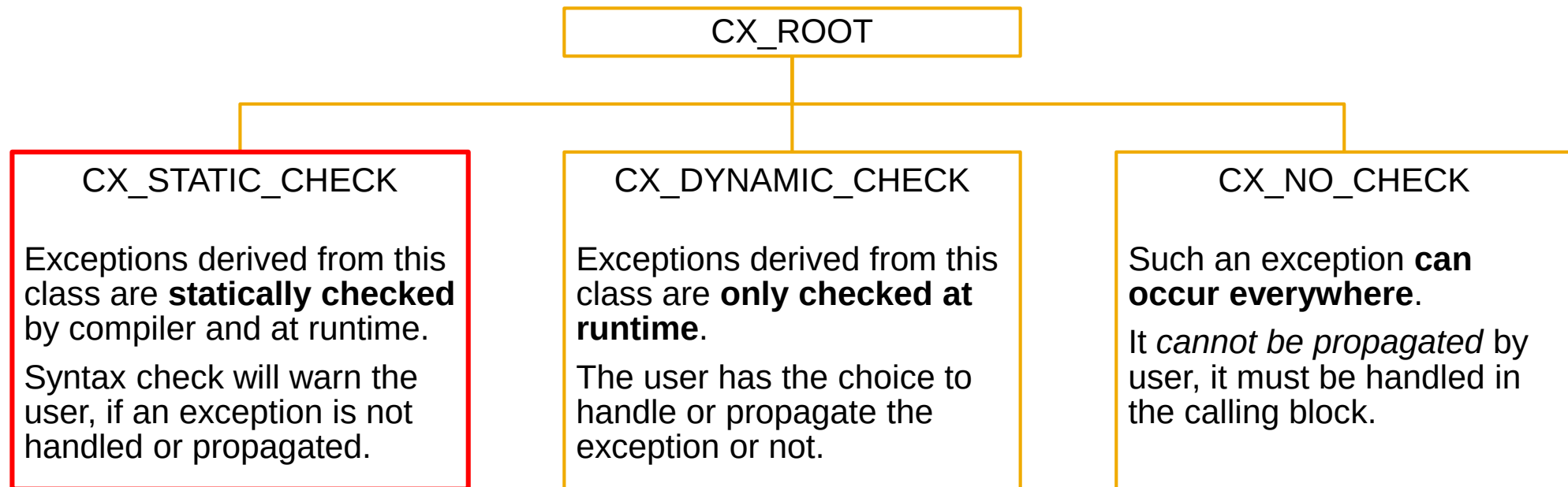


defined for specific exceptions
that only occur within one
single ABAP program

II. Exception classes

The exception hierarchy

- All exception classes must be derived from one of the classes CX_STATIC_CHECK, CX_DYNAMIC_CHECK or CX_NO_CHECK. These are subclasses of superclass CX_ROOT.
- Every exception class offers a range of standard methods and attributes. These are derived from the superclass CX_ROOT.



III. Exception handling

Basic concept

Statements need to be enclosed inside a TRY-ENDTRY structure

```
DATA lo_excp TYPE REF TO lcx_excp.
```

```
TRY.
```

```
* statement that might raise an exception
```

```
CATCH lcx_excp.
```

```
* OR
```

```
*CATCH lcx_excp INTO lo_excp.
```

```
*handler code for exception
```

```
ENDTRY.
```

The *exception handler* is defined by the specification of an exception class in the CATCH statement.

Optionally the *exception object* can be 'caught' and used for the exception handler.

- If an exception occurs, the ABAP Runtime checks if an appropriate handler is available (i.e. if the exception refers to the exception class (or its superclass) given in the CATCH statement).
- If no local handler is found, the system tries to propagate the exception to the calling program (read more about propagation later in this chapter).

IV. Exception raising

Two options

Two ways to raise a class based exception:

1. **create a new exception object** (= instance of the exception class <exception_class>)

```
RAISE EXCEPTION TYPE lcx_excp.  
* EXPORTING <attribute> = ...].
```

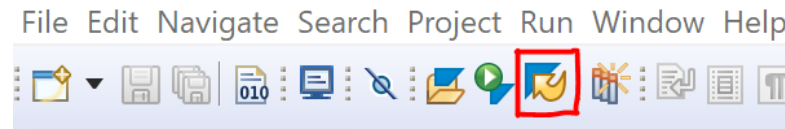
2. **make use of an existing exception object** lo_excp. This was either caught by a preceding CATCH statement or created by a CREATE OBJECT statement

```
DATA lo_excp TYPE REF TO lcx_excp.  
  
*CREATE OBJECT lo_excp. "OR  
*CATCH lcx_excp INTO lo_excp.  
  
RAISE EXCEPTION lo_excp.  
* [EXPORTING <attribute> = ...]
```

V. Global exception classes

Definition with the Class Builder

- Global exception classes are defined in the Class Builder in transaction SE24 which is embedded in the SAP GUI. To navigate to the SAP GUI in Eclipse, first click the icon for SAP GUI in the Menu Bar.



- At the creation of your exception class you have to select a superclass and decide whether a message class will be used.

Standardly exception classes are derived from CX_STATIC_CHECK. This assures the best syntax check and program stability

A screenshot of the 'Create Class ZCX_EXCP' dialog box in SAP. The dialog has a blue title bar with the text 'Create Class ZCX_EXCP' and a close button. It contains several fields: 'Class' with the value 'ZCX_EXCP', 'Superclass' with the value 'CX_STATIC_CHECK', 'Description' with the value 'Demo Exception Class', and 'Inst. Generation' with a dropdown menu set to 'Private'. Below these fields is a section titled 'Class Type' with five radio button options: 'Usual ABAP Class', 'Exception Class' (which is selected), 'With Message Class' (which has a checked checkbox), 'Persistent class', and 'Test Class (ABAP Unit)'. At the bottom left is a checkbox labeled 'Final'. At the bottom right are two buttons: a green 'Save' button and a red 'Cancel' button.

Since SAP NW AS 7.0, exception texts can be stored in message classes (< SAP NW AS 6.4: texts are stored in the Online Text Repository)

V. Global exception classes

Texts

- Similar to a standard ABAP class, an exception class provides attributes, texts and methods.
- The predefined attribute TEXTID points to the textual description of the error. The actual texts are customized in tab “Texts”.

Class Builder: Change Class ZCX_EXCP

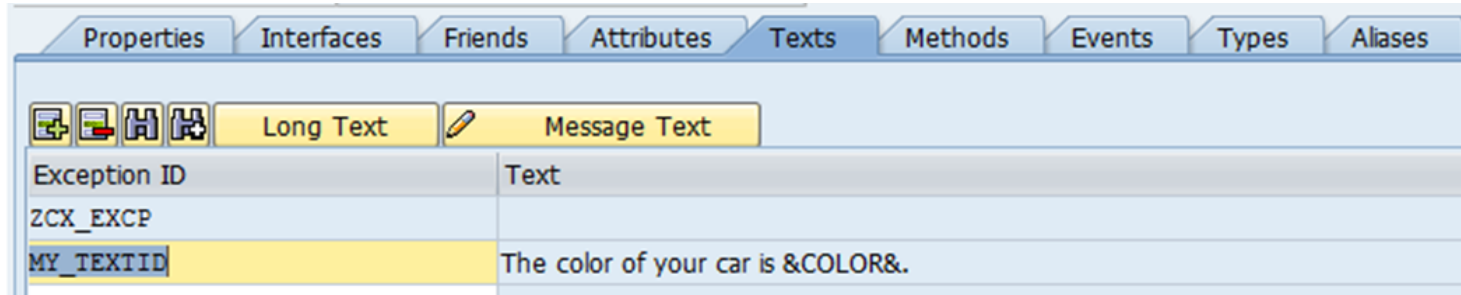
Class/Interface: ZCX_EXCP Implemented / Inactive

Properties Interfaces Friends **Attributes** Texts Methods Events Types Aliases

Attribute Level Visibility R... Typing Associated Type Description Initial value

IF_T100_MESSAGE~DEFAULT_TEXTID	Constant	Public	☐				
IF_T100_MESSAGE~T100KEY	Instance ...	Public	☐	Type	SCX_T100KEY		T100 Key with Paramete...
CX_ROOT	Constant	Public	☐	Type	SOTR_CONC		Exception ID: Value for ... '16AA9A3937A9BB56E1 ...
TEXTID	Instance ...	Public	☒	Type	SOTR_CONC		Key for Access to Messa...
PREVIOUS	Instance ...	Public	☒	Type Ref ...	CX_ROOT		Exception Mapped to th...
KERNEL_ERRID	Instance ...	Public	☒	Type	S380ERRID		Internal Name of Excepti...
IS_RESUMABLE	Instance ...	Public	☒	Type	ABAP_BOOL		Flag, Whether RESUME C...

Texts



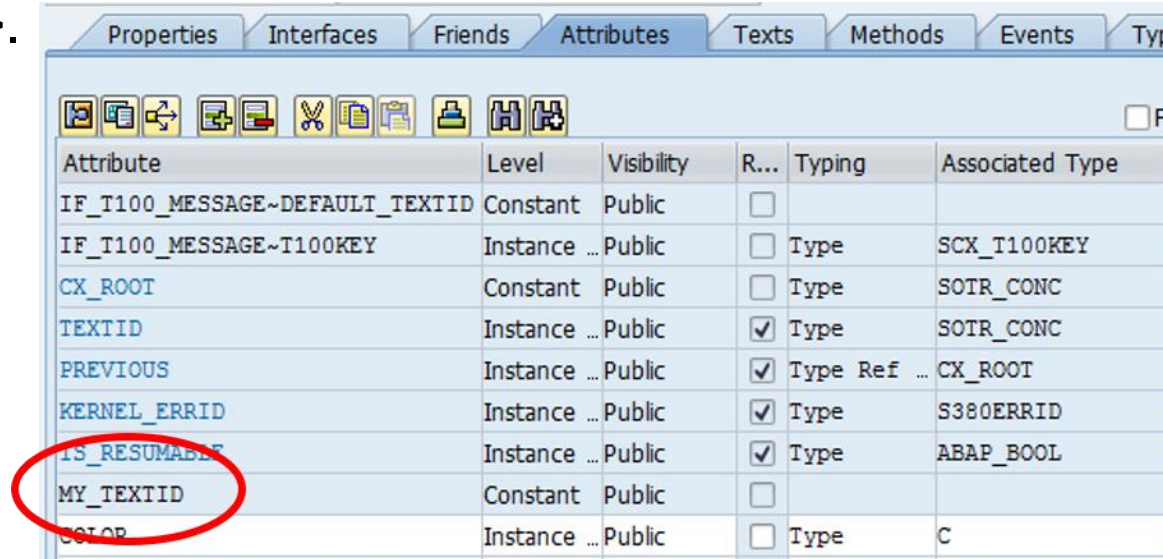
- On tab “Texts”, the textual description of the error can be defined.
- By default, the text with the Exception ID, which *equals the class name*, is displayed.
- Alternative texts can be defined by creating a new entry in column “Exception ID”. To display the alternative text, the related Exception ID needs to be specified in the RAISE statement.

```
RAISE EXCEPTION TYPE zcx_excp  
EXPORTING  
  textid = zcx_excp=>my_textid.
```

- The text can be stored either in a message class (since SAP NW AS 7.0) or in the OTR (Online Text Repository).

Text variables

- It is possible to insert variables into your exception text. For this, define an attribute `<my_text_variable>`.



Attribute	Level	Visibility	R...	Typing	Associated Type
IF_T100_MESSAGE~DEFAULT_TEXTID	Constant	Public	☐		
IF_T100_MESSAGE~T100KEY	Instance ...	Public	☐	Type	SCX_T100KEY
CX_ROOT	Constant	Public	☐	Type	SOTR_CONC
TEXTID	Instance ...	Public	☒	Type	SOTR_CONC
PREVIOUS	Instance ...	Public	☒	Type Ref ...	CX_ROOT
KERNEL_ERRID	Instance ...	Public	☒	Type	S380ERRID
IS_RESUMABLE	Instance ...	Public	☒	Type	ABAP_BOOL
MY_TEXTID	Constant	Public	☐		
COLOR	Instance ...	Public	☐	Type	C

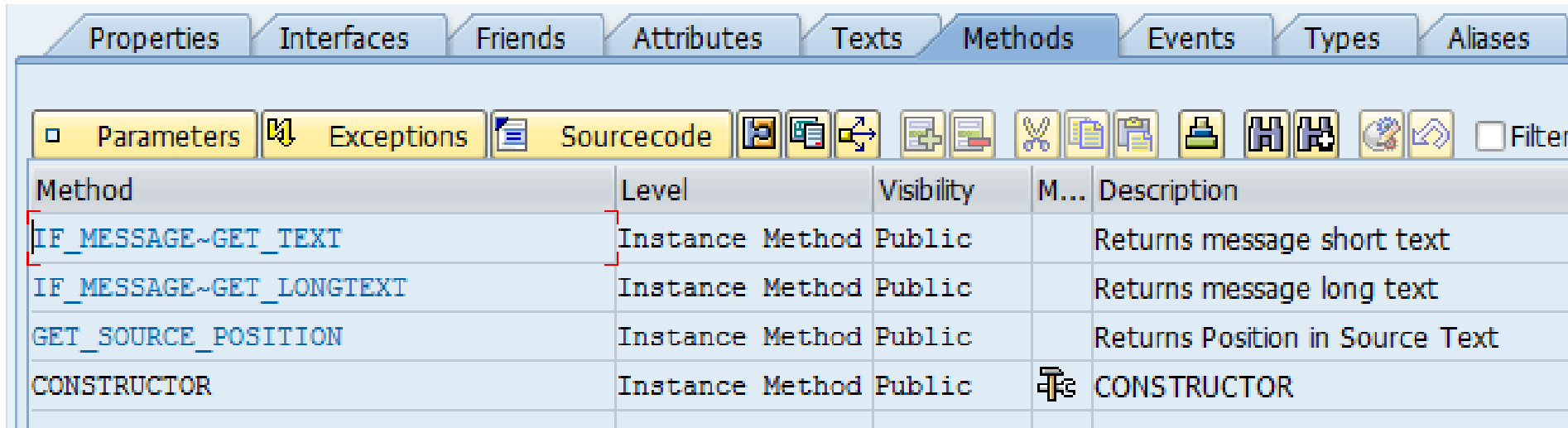
Insert the attribute as placeholder bracketed by `&...>&` (e.g. `&COLOR&`) into the exception text.

- Raise the exception with exporting parameters TEXTID and COLOR.

```
RAISE EXCEPTION TYPE zcx_excp
EXPORTING
  textid = zcx_excp=>my_textid
  color = 'BLUE'.
```

Predefined methods

- Every exception class has predefined methods inherited from CX_ROOT.



Method	Level	Visibility	M...	Description
IF_MESSAGE~GET_TEXT	Instance Method	Public		Returns message short text
IF_MESSAGE~GET_LONGTEXT	Instance Method	Public		Returns message long text
GET_SOURCE_POSITION	Instance Method	Public		Returns Position in Source Text
CONSTRUCTOR	Instance Method	Public	✚	CONSTRUCTOR

- **get_source_position**: The exception object returns the position where the exception has been raised.
- **get_text / get_longtext**: The exception object returns the textual description of the exception.

Now you can define your own global exception class with custom defined texts and attributes.

To consolidate your knowledge, you can do tasks 3 and 4 (and the challenge) of the ABAP Exception Handling exercises.

VI. Propagation of exceptions

Propagation

- Class-based exceptions that occur in procedures (methods, function modules, subroutines) can be propagated to the caller of the procedure.
- Exceptions can be propagated over several levels. The caller on the highest level must be enclosed in a TRY-ENDTRY structure.
- To enable propagation of an exception it must be published in the signature of the raising method using the RAISING addition.

```
DATA: lo_car TYPE REF TO lcl_car,  
      lo_excp TYPE REF TO lcx_excp.  
  
TRY.  
  
  CREATE OBJECT lo_car. ....  
  
  CATCH lcx_excp INTO lo_excp. ...  
  
ENDTRY.
```

The diagram illustrates the flow of an exception. A red dotted arrow originates from the `CREATE OBJECT lo_car.` statement in the `TRY` block and points to the `METHOD constructor.` block. Another red dotted arrow points from the `CALL METHOD get_tech_attr.` statement within the `constructor` block to the `METHOD get_tech_attr.` block. A final red dotted arrow points from the `RAISE EXCEPTION TYPE lcx_excp.` statement in the `get_tech_attr` block back to the `CATCH lcx_excp INTO lo_excp.` statement in the `TRY` block.

```
METHOD constructor.  
  CALL METHOD get_tech_attr. ....  
  *...  
ENDMETHOD.
```

```
METHOD get_tech_attr.  
  *source code to get technical attributes  
  RAISE EXCEPTION TYPE lcx_excp.  
  
ENDMETHOD.  
  
ENDCLASS.
```

Alternatives to program interruption

You may continue the program after having

- ignored the error
- showed a warning
- Writing the error to a log file
- corrected the error
-

Raise again the same or another exception and propagate it to the calling program.

Alternatives to program interruption

Correct the error and restart the program

- **RETRY:**

Jump back to the TRY statement after having caught the exception and removed the cause for the error (available since Release SAP NW 7.0 EhP 2).

```
DATA: lo_car TYPE REF TO lcl_car,  
      lo_excp TYPE REF TO lcx_excp.
```

```
TRY.
```

```
CREATE OBJECT lo_car.
```

```
CATCH lcx_excp INTO lo_excp.
```

```
*remove the error which raised the exception
```

```
RETRY.
```

```
ENDTRY.
```


Alternatives to program interruption

- **RESUME:**

Continue the program directly after the exception occurred (available since Release SAP NW 7.0 EhP 2).

```
DATA: lo_car TYPE REF TO lcl_car,  
      lo_excp TYPE REF TO lcx_excp.  
  
TRY.  
  
  CREATE OBJECT lo_car. ....  
  
  CATCH BEFORE UNWIND lcx_excp INTO lo_excp.  
  
    IF lo_excp->is_resumable = 'X'.  
  
      RESUME. ....  
  
    ENDIF.  
  
    *...  
  
  ENDTRY.
```

METHOD constructor.
CALL METHOD get_tech_attr.
*...
ENDMETHOD.

METHOD get_tech_attr.
*source code to get technical attributes
RAISE RESUMABLE EXCEPTION TYPE lcx_excp.
*continue program...
ENDMETHOD.

```
graph LR
    subgraph Constructor
        direction TB
        C1[METHOD constructor.]
        C2[CALL METHOD get_tech_attr. ....]
        C3[*...]
        C4[ENDMETHOD.]
    end
    subgraph GetTechAttr
        direction TB
        G1[METHOD get_tech_attr.]
        G2[*source code to get technical attributes]
        G3[RAISE RESUMABLE EXCEPTION TYPE lcx_excp.]
        G4[*continue program...]
        G5[ENDMETHOD.]
    end
    subgraph TryBlock
        direction TB
        T1[DATA: lo_car TYPE REF TO lcl_car, lo_excp TYPE REF TO lcx_excp.]
        T2[TRY.]
        T3[CREATE OBJECT lo_car. ....]
        T4[CATCH BEFORE UNWIND lcx_excp INTO lo_excp.]
        T5[IF lo_excp->is_resumable = 'X'.]
        T6[RESUME. ....]
        T7[ENDIF.]
        T8[*...]
        T9[ENDTRY.]
    end
    G3 -.-> T4
    T4 -.-> T6
    T6 -.-> C2
```

CATCH BEFORE UNWIND
The context in which the exception was raised (including all called procedures and their local data) is deleted only after exiting the CATCH block.

Check your knowledge

Check your knowledge

- Standardly, exception classes are defined as subclasses of CX_STATIX_CHECK.
☐ True ☐ False
- Exception classes can only be defined globally in the Class Builder.
☐ True ☐ False
- Explain the difference between an exception class and an exception object.
- Which of the following statements is used to jump back to the TRY statement in a TRY-ENDTRY structure?
☐ RETRY ☐ CATCH ☐ TRY-ENDTRY ☐ RAISE

Solution

Solution

- Standardly, exception classes are defined as subclasses of CX_STATIC_CHECK.

☐ True ☒ False

- Exception classes can only be defined globally in the Class Builder.

☐ True ☒ False

- Explain the difference between an exception class and an exception object.

See section *Introduction – What is an exception?*

- Which of the following statements is used to jump back to the TRY statement in a TRY-ENDTRY structure?

☒ RETRY ☐ CATCH ☐ TRY-ENDTRY ☐ RAISE

References

- BC401 ABAP Objects, Teilnehmerhandbuch. Version der Schulung: 92, 2011

© 2018 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see <http://global12.sap.com/corporate-en/legal/copyright/index.epx> for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.